Acquisitions API

Architecture Explainer

Codebase architecture and how it works

Generated: May 18, 2026

1. Identify the Big Picture

What type of project is this?

A Node.js REST API named Acquisitions. It is a backend HTTP service (not a full web app or CLI). Clients call it over JSON; there is no frontend in this repository.

What problem does it solve?

It is meant to back an acquisitions product (tracking deals, companies, or similar). Today the repository is an auth foundation: user registration, with sign-in and sign-out stubbed. The workspace path suggests Docker containerization may follow; there is no Dockerfile in the repo yet.

2. Core Architecture

Style

Layered monolith — one Express app, one process, with clear separation of concerns: routes !’ controllers !’ services !’ database.

High-level flow

```
HTTP Client
    |
    v
Express App (helmet, cors, json, cookies, morgan, Arcjet)
    |
    +-- /, /health, /api
    +-- /api/auth/*  -->  Routes --> Controllers --> Services --> Neon Postgres
```

Folder layout

Path	Role
src/index.js	Entry: imports server.js
src/server.js	Binds HTTP port
src/app.js	Express setup, middleware, routes
src/routes/	URL to handler wiring
src/controllers/	HTTP: validate, status codes, responses
src/services/	Business logic (passwords, JWT, DB)
src/models/	Drizzle schema (Postgres tables)
src/validations/	Zod request schemas
src/middleware/	Cross-cutting (Arcjet security)
src/config/	DB, logger, Arcjet client
drizzle/	Generated SQL migrations

3. Key Components

Application shell (app.js)

- Helmet — security headers
- CORS — cross-origin requests
- express.json / urlencoded — body parsing
- cookie-parser — reads token cookie
- Morgan — HTTP logs forwarded to Winston
- securityMiddleware — Arcjet rate limit, bots, shield

Auth routes

Method	Path	Status
POST	/api/auth/signup	Implemented
POST	/api/auth/signin	Stub
POST	/api/auth/signout	Stub

Auth service

- hashPassword / comparePassword — bcryp
- generateToken — JWT with id, email, role (
- createUser — duplicate check, hash, insert

Data layer

database.js connects Neon serverless driver witl
name, email, password, role, created_at, update

Security (Arcjet)

Global rules: shield, bot detection (allows search
Per-request middleware adds role-based limits: g
yet, so everyone is treated as guest.

4. Data Flow & Communication

Signup flow (implemented)

```
Client -> Express -> securityMiddleware -> signup controller
  -> Zod validation -> createUser service
  -> SELECT email -> INSERT user -> generateToken
  -> Set-Cookie + 201 JSON response
```

Discovery endpoints

- GET / — welcome message
- GET /health — status, timestamp, uptime
- GET /api — API alive check

No service-to-service calls. Only client !” API !” database and API !” Arcjet (in-process SDK).

5. Tech Stack & Dependencies

Technology	Role
Node.js (ES modules)	Runtime
Express 5	HTTP server, routing, middleware
Drizzle ORM	Schema, queries, migrations
Neon serverless	Postgres over HTTP
Zod	Request validation
bcrypt	Password hashing
jsonwebtoken	Auth tokens
cookie-parser	Cookie-based session token
Winston + Morgan	Logging
Helmet + CORS	HTTP hardening
Arcjet	Rate limiting, bots, attack shield
dotenv	Environment configuration

6. Execution Flow — User Signup

- Client sends POST /api/auth/signup with JSON body.
- Request passes Helmet, CORS, parsers, cookie-parser, Morgan.
- securityMiddleware runs Arcjet; may return 403 or 429.
- authRouter delegates to signup controller.
- signupSchema.safeParse — on failure returns 400.
- createUser checks email, hashes password, inserts row.
- generateToken builds JWT; cookies.set sets HTTP-only cookie.
- Response: 201 with message, user object, and token.

Example request

```
POST /api/auth/signup
Content-Type: application/json

{
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "secret12",
  "role": "user"
}
```

Example success response

```
HTTP/1.1 201 Created
Set-Cookie: token=<jwt>; HttpOnly; SameSite=Strict

{
  "message": "User registered successfully.",
  "user": { "id": 1, "name": "Jane Doe", ... },
  "token": "<jwt>"
}
```

7. Strengths & Tradeoffs

Strengths

- Clear layering — easy to add acquisition routes beside auth.
- Import maps — clean internal imports without deep relative paths.
- Schema + migrations — Drizzle keeps DB and code in sync.
- Security-minded defaults — Helmet, Arcjet, bcrypt, HTTP-only cookies.
- Observability — Winston, Morgan, dedicated /health endpoint.
- Serverless-friendly DB — Neon HTTP driver fits edge deploys.

Limitations & watch-outs

- Incomplete auth — sign-in/sign-out are placeholders; no auth middleware.
- req.user never set — role-based Arcjet limits always use guest.
- Duplicate JWT paths — auth.service (1h) vs utils/jwt.js (1d).
- createUser catch block may mask EMAIL_ALREADY_EXISTS errors.
- Security middleware — audit return after every Arcjet denial.
- No Docker yet — containerization may be planned for course.
- Acquisitions domain — no acquisition entities/routes yet.

8. Final Summary

Acquisitions is a small Express 5 REST API structured as routes !’ controllers !’ services !’ Drizzle/Neon, focused today on user signup with Zod validation, bcrypt, JWT + cookies, and Arcjet rate limiting and bot protection. It is a monolithic backend ready to grow with acquisition-specific endpoints once auth (sign-in, JWT middleware, req.user) is finished.

Elevator pitch

It is a Node/Express API for an acquisitions product, wired to Neon Postgres through Drizzle. Right now it only fully implements signup with JWT cookies and Arcjet security; sign-in and the actual acquisitions features are still to come. Think of it as the secured auth shell you will hang the rest of the API on.